Q1 – Handling Failure in a Bank ATM Simulation

Shared: Failure points and classification

#	Failure point	Exception / problem	Category	Caught where
1	User types letters for amount	`InputMismatchException` (or `NumberFormatException` if `Integer.parseInt`)	Unchecked	`catch` in `withdraw()` caller
2	Negative or zero amount	`IllegalArgumentException` (thrown by us)	Unchecked	`catch` in main
3	Amount > balance	`InsufficientFundsException` (user-defined)	Checked	`catch` in main
4	Card-reader file missing	`FileNotFoundException`	Checked (subclass of `IOException`)	`catch` in main
5	Account number not found / null	`NullPointerException`	Unchecked	Should be prevented, caught as last resort
6	Memory exhausted / JVM failure	`OutOfMemoryError`	Error	Not caught

Code

```
import java.io.*;
import java.util.*;

class InsufficientFundsException extends Exception {
    InsufficientFundsException(String msg) { super(msg); }
}

public class ATM {
```

```
    static double balance = 5000;

    static void withdraw(double amt) throws InsufficientFundsException
{
        if (amt <= 0) throw new IllegalArgumentException("Amount must
be positive");
        if (amt > balance) throw new
InsufficientFundsException("Balance " + balance + " < " + amt);
        balance -= amt;
    }

    static void readCard(String file) throws FileNotFoundException {
        new FileReader(file).toString();   // throws if missing
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Card file: ");
            readCard(sc.next());
            System.out.print("Amount: ");
            double amt = sc.nextDouble();            //
InputMismatchException
            withdraw(amt);
            System.out.println("Dispensed " + amt + ", balance " +
balance);
        } catch (FileNotFoundException e) {
            System.out.println("Card reader error: " +
e.getMessage());
        } catch (InputMismatchException | IllegalArgumentException e)
{   // multi-catch
            System.out.println("Invalid input: " + e);
        } catch (InsufficientFundsException e) {
            System.out.println("Declined: " + e.getMessage());
        } catch (Exception e) {
// last resort
            System.out.println("Unexpected: " + e);
        } finally {
            System.out.println("Card returned. Session closed.");
            sc.close();
        }
    }
}
```

Student A – Exception design

Hierarchy (relevant part):

```
Throwable
├── Error
```

```
│   └── OutOfMemoryError
└── Exception
    ├── IOException
    │   └── FileNotFoundException          (checked)
    ├── InsufficientFundsException         (checked, user-defined)
    └── RuntimeException                   (unchecked)
        ├── IllegalArgumentException
        │   └── NumberFormatException
        ├── InputMismatchException
        └── NullPointerException
```

Catch order justification: Java checks catch blocks top to bottom and runs the first one that matches. Specific subclasses must come before their superclasses; otherwise the compiler reports "exception already caught" because the later block is unreachable. In the code, `FileNotFoundException` and the multi-catch come first and `Exception` is last as a safety net. In a multi-catch the alternatives must not be related by inheritance.

What `finally` guarantees: It runs whether the try completes normally, an exception is caught, or an exception is uncaught and propagates. Here it guarantees the card is returned and the `Scanner` is closed on every path. The exceptions are `System.exit()`, JVM crash or a killed thread.

Student B – Test table (run and fill the last columns)

#	Input	Path triggered	Catch executed	finally ran?
1	card=missing.txt	File missing	`FileNotFoundException`	Yes
2	card=ok.txt, amount=abc	Letters	multi-catch (`InputMismatchException`)	Yes
3	amount=-500	Negative	multi-catch (`IllegalArgumentException`)	Yes
4	amount=9000	Over balance	`InsufficientFundsException`	Yes
5	amount=1000	Valid	none	Yes
6	(remove the catch for the file)	Uncaught	none, JVM prints trace	Yes (runs before the thread dies)

Stack trace to explain (for an uncaught exception, e.g. `Integer.parseInt("abc")`):

```
Exception in thread "main" java.lang.NumberFormatException: For input
string: "abc"
    at
java.base/java.lang.NumberFormatException.forInputString(NumberFormatE
xception.java:67)
    at java.base/java.lang.Integer.parseInt(Integer.java:661)
    at ATM.main(ATM.java:25)
```

- Line 1: thread name, exception class and message.
- The first `at` line is where the exception was created (the deepest frame).
- Following lines walk back up the call stack (the caller of each method).
- The last line is your own code, the line you need to fix.

Broader catch before narrower: putting `catch (Exception e)` before `catch (FileNotFoundException e)` gives a compile-time error: `exception FileNotFoundException has already been caught`. Show the screenshot.

Joint design question

Catching everything with one `catch (Exception e)` is acceptable only at a top-level boundary (e.g. main loop or request handler) to log and show a generic message so the program doesn't crash. It's not acceptable inside business logic because you lose: different recovery per failure (retry on bad input vs. abort on missing card), meaningful user messages, compiler help in spotting unhandled checked exceptions, and you may swallow programming bugs such as NPE.

Alternative concept and complexity

Error codes version:

```java
static int withdraw(Scanner sc) {
    if (!sc.hasNextDouble()) return -1;       // not a number
    double amt = sc.nextDouble();
    if (amt <= 0) return -2;
    if (amt > balance) return -3;
    balance -= amt; return 0;
}
// caller: switch (withdraw(sc)) { case -1: ... case -2: ... }
```

- **Readability:** exceptions keep the normal path clean and put error handling in one place. Error codes add an `if` after every call, and callers can silently ignore the codes.
- **Checks:** error codes need one explicit check per failure point per call (k checks every call, even on success). Exceptions cost zero checks on the success path.
- **Time:** both are O(1) logically. But creating and throwing an exception is expensive in practice: `fillInStackTrace()` walks the stack, costing O(d) where d = stack depth. An error code is a plain integer return, O(1) with no allocation.
- **Space:** exception object holds the stack trace, O(d). An error code is O(1).
- **Conclusion:** use exceptions for exceptional events and codes (or validation) for expected, frequent failures such as bad user input in a hot loop.